

Part A – Answers

1. c) CREATE
 2. b) CREATE DATABASE
 3. c) TRUNCATE
 4. b) DROP
 5. b) RENAME TABLE old_name TO new_name;
 6. a) Add new rows to a table
 7. b) Modify existing data
 8. b) DELETE
 9. a) SELECT
 10. d) INSERT
-

Part B – Answers

1. Create a database named CompanyDB

```
CREATE DATABASE CompanyDB;
```

2. Create Employee table

```
CREATE TABLE Employee (  
    EmpID INT,  
    EmpName VARCHAR(50),  
    Salary DECIMAL(10,2)  
);
```

3. Insert multiple records

```
INSERT INTO Employee (EmpID, EmpName, Salary)  
VALUES  
(101, 'Ravi', 50000),  
(102, 'Priya', 45000),  
(103, 'Arjun', 60000);
```

4. Update Ravi's salary to 55000

```
UPDATE Employee  
SET Salary = 55000
```

WHERE EmpName = 'Ravi';

5. Increase salary of all employees by 10%

```
UPDATE Employee  
SET Salary = Salary * 1.10;
```

6. Delete employee where EmpID = 103

```
DELETE FROM Employee  
WHERE EmpID = 103;
```

7. Delete all records but keep table structure

```
TRUNCATE TABLE Employee;
```

8. Remove Employee table completely

```
DROP TABLE Employee;
```

9. Rename Employee table to Staff

```
RENAME TABLE Employee TO Staff;
```

10. Drop CompanyDB database

```
DROP DATABASE CompanyDB;
```

11. Display names and salaries where salary > 50000

```
SELECT EmpName, Salary  
FROM Employee  
WHERE Salary > 50000;
```

12. Display total salary of each department

```
SELECT Department, SUM(Salary) AS TotalSalary  
FROM Employee  
GROUP BY Department;
```

13. Display departments where average salary > 60000

```
SELECT Department, AVG(Salary) AS AverageSalary
FROM Employee
GROUP BY Department
HAVING AVG(Salary) > 60000;
```

14. Display first 5 rows

```
SELECT *
FROM Employee
LIMIT 5;
```

15. Difference between Primary Key and Foreign Key

Primary Key	Foreign Key
Uniquely identifies each record in a table.	Links one table to another.
Cannot contain NULL values.	Can contain NULL values (unless restricted).
Must contain unique values.	Duplicate values are allowed.
Only one primary key per table.	A table can have multiple foreign keys.
Ensures entity integrity.	Ensures referential integrity.

Joins

1. INNER JOIN

Returns only the matching records from both tables.

```
SELECT *
FROM Employee
INNER JOIN Department
ON Employee.DeptID = Department.DeptID;
```

2. LEFT JOIN

Returns all records from the left table and matching records from the right table.

```
SELECT *  
FROM Employee  
LEFT JOIN Department  
ON Employee.DeptID = Department.DeptID;
```

3. RIGHT JOIN

Returns all records from the right table and matching records from the left table.

```
SELECT *  
FROM Employee  
RIGHT JOIN Department  
ON Employee.DeptID = Department.DeptID;
```

4. FULL OUTER JOIN

Returns all matching and non-matching records from both tables.

```
SELECT *  
FROM Employee  
FULL OUTER JOIN Department  
ON Employee.DeptID = Department.DeptID;
```

5. CROSS JOIN

Returns the Cartesian product of both tables.

```
SELECT *  
FROM Employee  
CROSS JOIN Department;
```

6. SELF JOIN

Joins a table with itself.

```
SELECT A.EmpName AS Employee,  
       B.EmpName AS Manager  
FROM Employee A  
JOIN Employee B  
ON A.ManagerID = B.EmpID;
```

